

第 7 章（树）作业：补充作业题的参考答案（by ckw）

1、答：求二叉树 b 中第 k 层的结点数，设计了三个不同于教材上的算法，如下：

（1）采用先序遍历方法求解，有两种算法，算法 1 最简单，代码如下：

```
int Lnodenum1(BTNode *b, int k) //求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);
  else //求左右子树第 k-1 层节点个数之和
    return( Lnodenum2(b->lchild,k-1) + Lnodenum2(b->rchild,k-1) );
}
```

（2）采用先序遍历方法求解的算法 2，代码如下：

```
int Layer = 1; //全局变量 L 记录当前访问的结点层次号
void nodeNum(BTNode *b, int k, int &n) //采用先序遍历方法 2 求解
{ if(b!=NULL)
  { if(Layer==k)
    { n++;
      Layer++;
      nodeNum(b->lchild, k, n);
      nodeNum(b->rchild, k, n);
      Layer--;
    }
  }
}
int Lnodenum2(BTNode *b,int k) //求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);
  int n=0;
  nodeNum(b, k, n); //调用上面的函数
  return n;
}
```

（3）采用分层次的层次遍历方法求解，算法代码如下：

```
int Lnodenum3(BTNode *b, int k) //求二叉树 b 中第 k 层的结点数
{ if (b==NULL || k<1) //空树,返回 0
    return(0);
  else if(k==1) //第一层只有一个根结点
    return(1);
  BTNode *p, *que[MaxSize]; //定义结点指针和顺序队列
  int f=-1, r=-1; //初始化队头和队尾指针
  int lev=1; //当前层次为第 1 层
  r=(r+1)%MaxSize;
  que[r] = b; //根结点入队
  while(f!=r) //队列非空时循环
  { int num=(r-f+MaxSize)%MaxSize; //求当前层次的结点数（即队列长度）
    if(lev==k) return num; //当前层为第 k 层，返回该层结点数
    for(int i=0; i<num; i++)
    { f=(f+1)%MaxSize;
      p = que[f]; //出队结点 p
    }
  }
}
```

```

        if(p->lchild!=NULL)
        { r=(r+1)%MaxSize;
          que[r] = p->lchild;    //有左孩子，将其入队
        }
        if(p->rchild!=NULL)
        { r=(r+1)%MaxSize;
          que[r] = p->rchild;    //有右孩子，将其入队
        }
    }
    lev++;    //当前层访问完毕，进入下一层处理
}
return 0; // 参数 k 超出了树的高度
}

```

2、答：（此题有两种算法设计思路，如下）

算法 1 设计思路：首先采用递归的先序遍历方法查找值为 y 的结点（结点 y），返回指向结点 y 的指针；再采用递归方法求结点 x 在以结点 y 为根的子树中的层次 k，那么 x 属于 y 的第 k-1 代子孙。所以，分别定义两个递归调用的函数 `BTNode * FindNode(BTNode *b, char x)` 和 `int Level(BTNode *b, char x)` 来实现。通过调用这两个函数，可以求得二叉树 b 中的结点 x 属于结点 y 的第几代子孙。

对应的**算法代码**如下：

```

BTNode * FindNode(BTNode *b, char x) //在二叉树 b 中查找值为 x 的结点
{ BTNode *p;
  if (b==NULL) return NULL;
  if(b->data == x) return b;
  p=FindNode(b->lchild, x); //在左子树中查找
  if(p==NULL) //在左子树中未找到，再在右子树中查找
    p=FindNode(b->rchild, x);
  return p;
}

```

```

int Level(BTNode *b, char x) //求值为 x 的结点在二叉树 b 中的层次
{ int h;
  if (b==NULL) return 0;
  if(b->data == x) return 1;    //x 处于根结点，所在层次为 1.
  h=Level(b->lchild, x); //求 x 在左子树中的层次
  if(h==0) //在左子树中未找到 x，再求 x 在右子树中的层次
    h=Level(b->rchild, x);
  return h!=0?(h+1):0;
}

```

`int descendant(BTNode *b, char x, char y)` //求二叉树 b 中的结点 x 属于结点 y 的第几代子孙

```

{ BTNode *p;
  int gen;
  p= FindNode(b,y); //在二叉树 b 中查找 y
  gen = Level(p, x); //求 x 在子二叉树 p 中的层次
}

```

```

return gen>0?(gen-1):0;
}

```

算法2 设计思路：在二叉树中找结点 y 和求结点 x 的层次，可以用同一个递归函数来实现。采用先序遍历方法同时查找结点 x 和 y 。如果先找到结点 y ，就求解结点 x 在 y 的左右子树中的层次，若层次非零值，它就表示 x 属于 y 的第几代，否则， x 不在 y 的子树中，即 x 不属于 y 的后代。如果先找到 x ，那么 x 不可能是 y 的后代，直接返回 0。因此，需要对求二叉树中结点层次的算法进行修改。算法代码如下：

//下面的函数：求二叉树 b 中结点 x 在结点 y 的左右子树中的层次。

```

int Levelx (BTNode *b,ElemType x,ElemType y,int h)  // (h 置初值 1)
{ int k;
  if (b==NULL)
    return(0);
  else if (b->data==y && x!=y) //先找到 y
  { k = Levelx(b->lchild,x,x,1); //查找 x 在左子树中的层次（忽略 y，h 重置初值
1)
    if (k!=0)          //找到了 x
      return(k);      //返回 x 在子树中的层次
    else              //在左子树中未找到 x，再在右子树中查找 x 的层次（忽略 y，h 重置
初值 1)
      return( Levelx(b->rchild,x,x,1) );
  }
  else if (b->data==x)
  { if(x!=y) //先找到 x
    return(0); //返回 0，表示 x 不是 y 的后代
    else    //后找到 x
      return(h); //返回 x 所在的层次。
  }
  else //当前结点值既不是 y 也不是 x
  { k = Levelx(b->lchild,x,y,h+1); //在左子树中查找 x 和 y
    if (k==0) //在左子树中未找到 x，再在右子树中查找 x 和 y
      k = Levelx(b->rchild,x,y,h+1);
    return(k);
  }
}

```

//下面的函数：求二叉树 b 中的结点 x 属于结点 y 的第几代子孙

```

int descendant(BTNode *b,ElemType x,ElemType y)
{ return( Levelx(b,x,y,1) );
}

```

3、答：a) 代码中各空白处填写内容如下：

① struct tnode *lchild, *rchild

② front = rear = -1 或 front=-1; rear=-1

- ③ `front != rear` 或 `front < rear`
- ④ `front++` 或 `front=front+1`
- ⑤ 队首元素出队
- ⑥ `rear++` 或 `rear = rear+1`
- ⑦ 左孩子指针入队
- ⑧ `Qu[rear].father=front`
- ⑨ `rear++` 或 `rear = rear+1`
- ⑩ 右孩子指针入队
- ⑪ `Qu[rear].father=front`
- ⑫ `b->lchild ==NULL && b->rchild ==NULL`
- ⑬ `Qu[np].node->data`
- ⑭ `pathLen > maxLen`

b) 顺序队，层次遍历. (计 2 分)

c) 算法的时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$.